MSE 312: MECHATRONICS DESIGN II

Introduction to Simulink for Control Design and Simulation

OBJECTIVES

To become familiar with the Simulink environment and use it to design and test a feedback controller.

Required Components/Material:

- Matlab/Simulink

References:

https://www.mathworks.com/help/simulink/getting-started-with-simulink.html
http://ctms.engin.umich.edu/CTMS/index.php?aux=Basics_Simulink

BASICS

Simulink® is a block diagram environment for multi domain simulation and Model-Based Design. Simulink provides a graphical editor, customizable block libraries, and solvers for modeling and simulating dynamic systems.

To start Simulink, click on the Simulink button at the top of the MATLAB window as shown here:

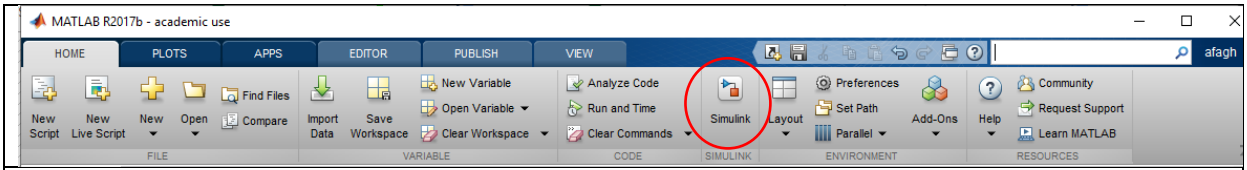


Figure 1. Opening Simulink

When it starts, Simulink brings up a single window, entitled Simulink Start Page which can be seen here.

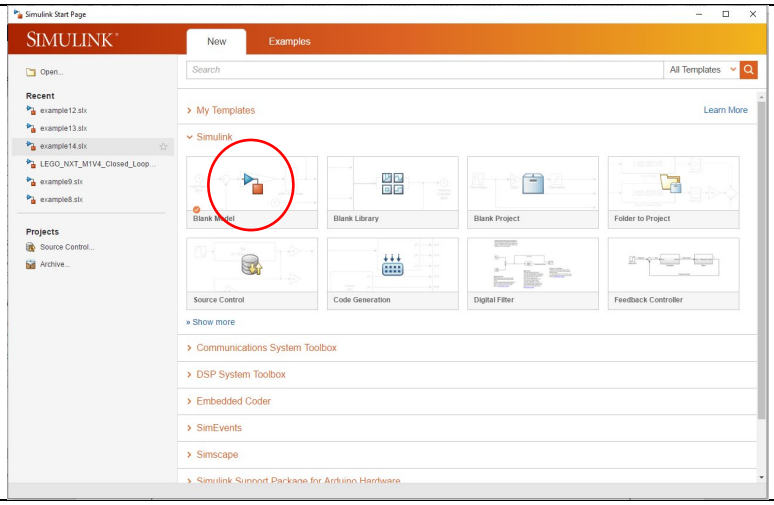


Figure 2. Opening a blank Simulink model

Once you click on Blank Model, a new window will appear as shown below.

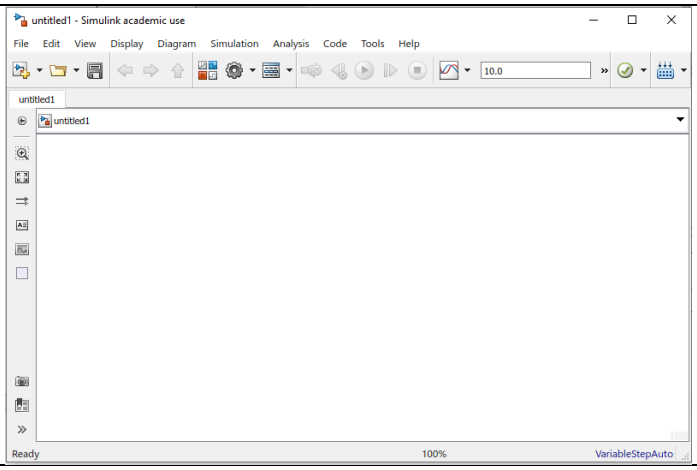


Figure 3. A new Simulink model

In this window, click on the Library Browser as marked in the next figure and you will see the Simulink library browser window where the blocks are. Blocks are used to generate, modify, combine, output, and display signals. There are several general classes of blocks within the Simulink library, some of the most commonly used are:

- **Sources:** Used to generate various signals to use as inputs to the models
- **Sinks:** Uused to output or display signals (e.g. an oscilloscope)
- **Continuous:** Continuous-time system elements (transfer functions, state-space models, PID controllers, etc.)
- **Discrete:** Linear, discrete-time system elements (discrete transfer functions, discrete state-space models, etc.)
- **Math Operations:** Contains common math operations (gain, sum, product, absolute value, etc.)
- **Ports & Subsystems:** Contains useful blocks to build a system

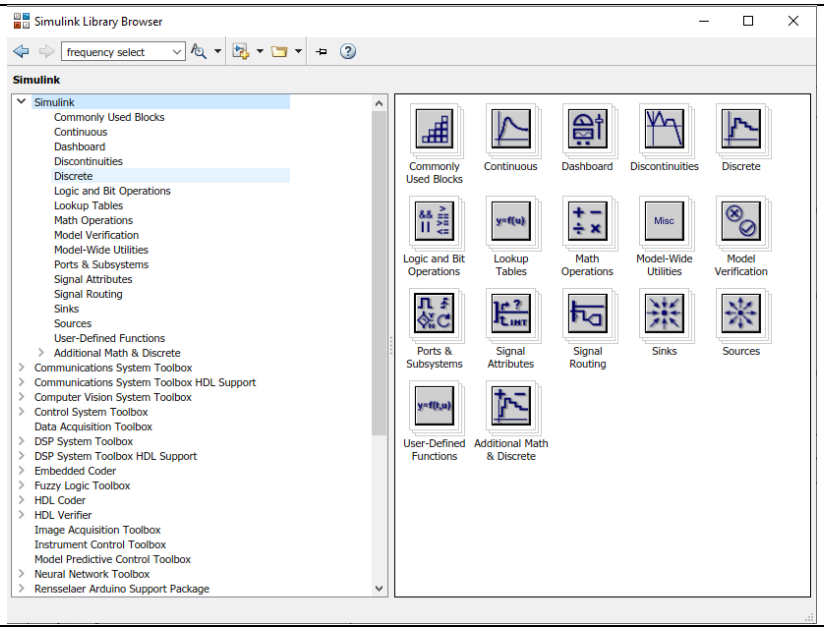


Figure 4. Simulink Library Browser

To use any of these blocks, find it in the left hand side list and drag and drop or copy and paste it onto the blank model. You can also search for a specific block on top of the left hand menu if you don't know where to find it. The inputs and outputs of blocks can be connected to each other by dragging a line from the output port of one block to the input of another. These lines represent the transmission of signals between blocks.

Example 1: Create a simple first order model and simulate it in time domain
Let's simulate a body of mass m moving on a surface with the friction coefficient b driven by an outside force $f(t)$ as shown in the following figure.

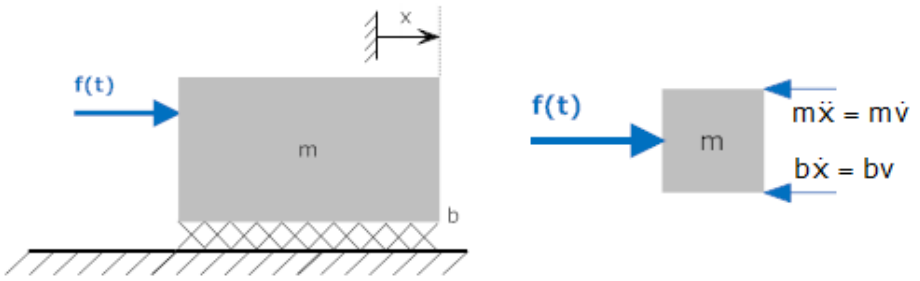


Figure 5. A mass moving on a surface with friction.

The first step is to derive the transfer function between the input and output of the system (force f and speed v). By writing the differential equation governing the motion and taking the Laplace transform, the transfer function is obtained as:

$$M\dot{v} + bv = f(t)$$
$$msV(s) + bV(s) = F(s)$$
$$\frac{V(s)}{F(s)} = H(s) = \frac{1}{ms + b} = \frac{\frac{1}{b}}{1 + \frac{m}{b}s} = \frac{k}{1 + Ts}$$

In the above equation, $1/b$ is called the static gain and $T = m/b$ is called the time constant.

Let's simulate the system when $m = b = 1$ and obtain the response in Simulink.

First, use a transfer function block from *Simulink>continuous* as shown in the following figure:

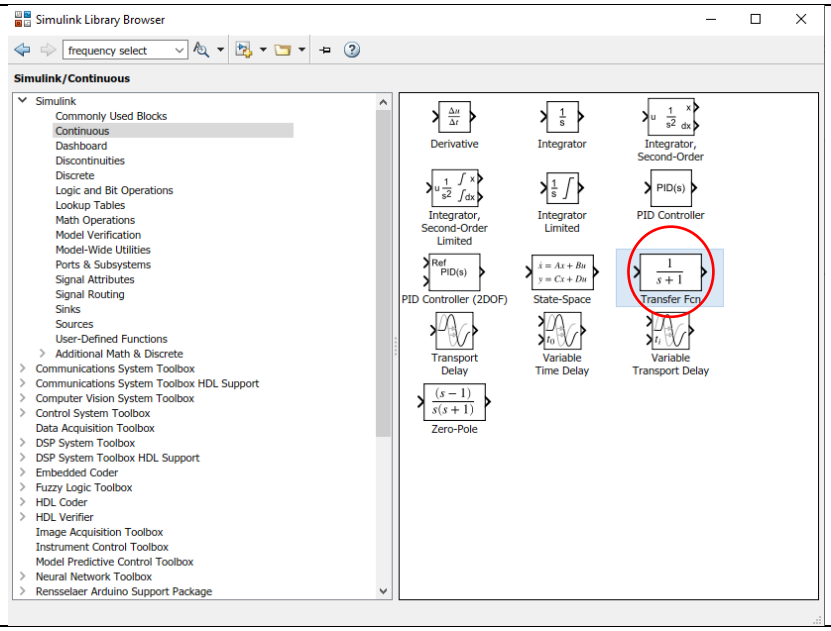


Figure 6. Transfer function block

Drag and drop the block onto your model and double click on it to see the following block parameters

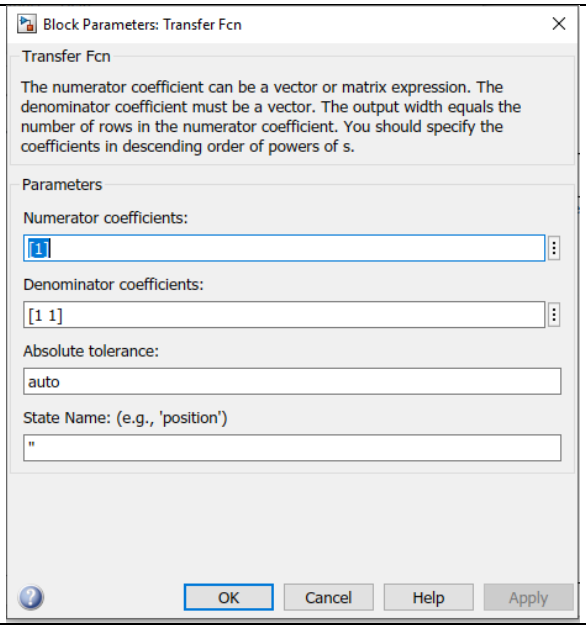


Figure 7. Transfer function block parameters

The parameters of the transfer function can be inserted in the form of coefficient vectors for numerator and denominator polynomials of the transfer function. For example:
[1] means 1
[1 1] means $s + 1$
[0 1] means s
[1 1 1] means $s^2 + s + 1$.

Next, drag and drop a step block from *simulink>sources* and a scope block from *simulink>sinks* as shown below

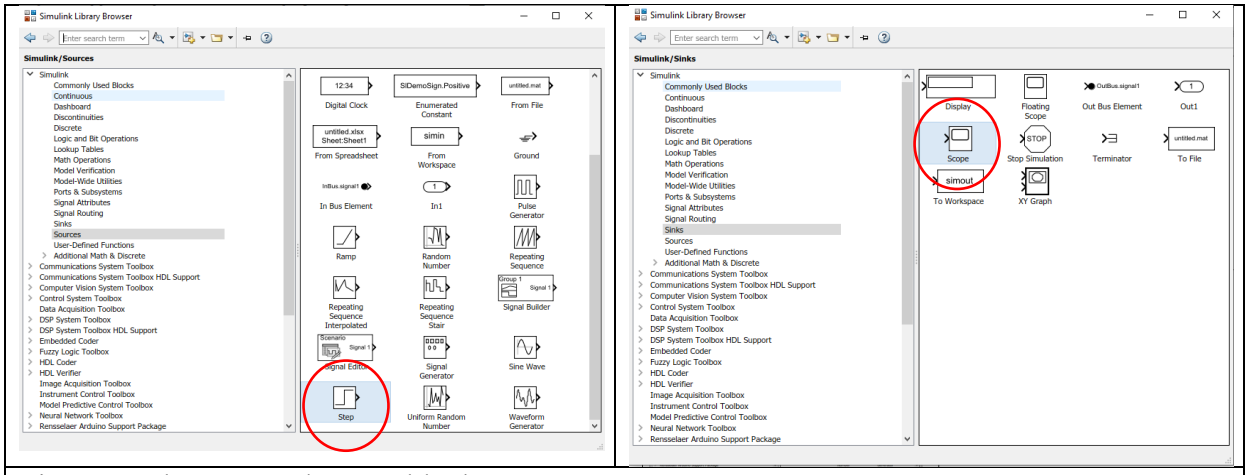


Figure 8. The Step and Scope blocks

Connect the three blocks as shown below

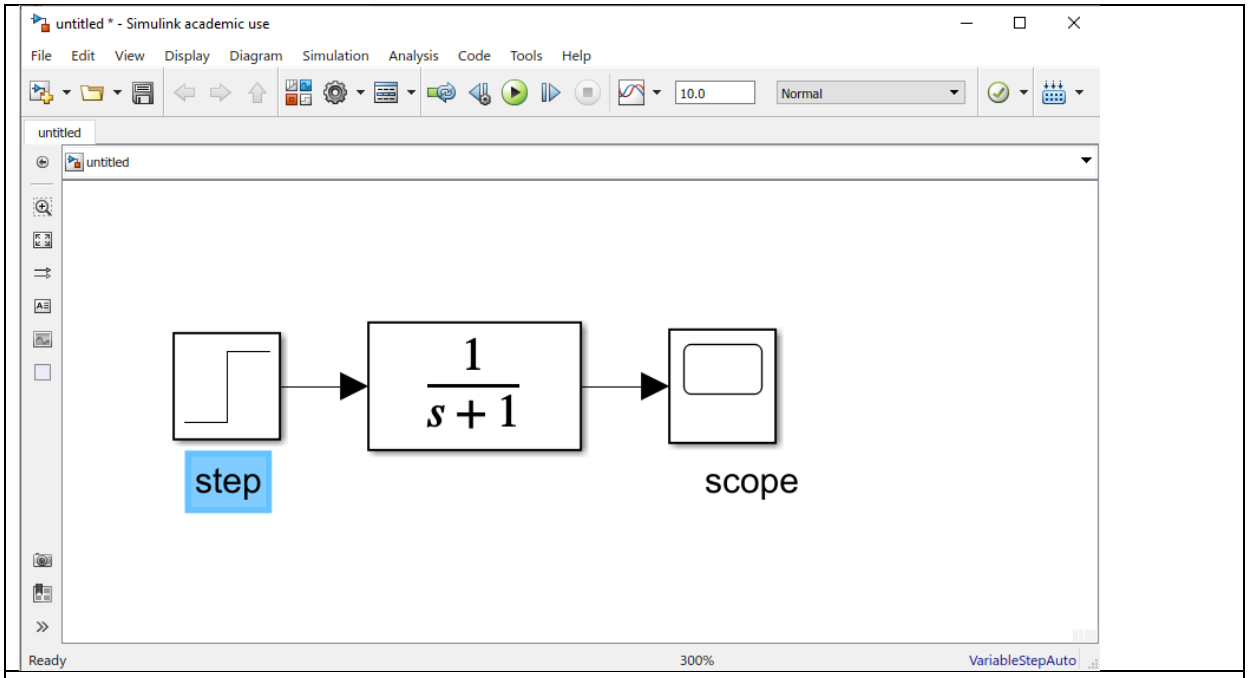


Figure 9. First order system simulation model

By clicking on the play icon on top of this window, you can run the simulation. But before that, let's configure the simulation parameters:
In the model window, select the gear icon representing the Model Configuration Parameters from the Simulation menu. You will see the following dialog box.

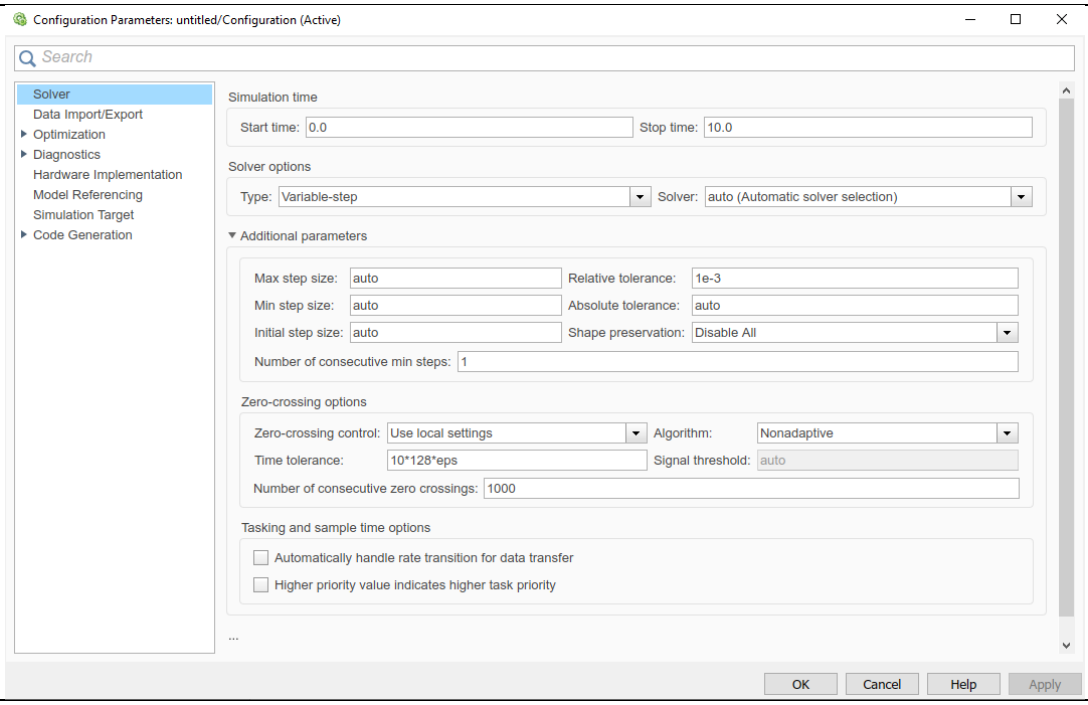


Figure 10. Simulink model configuration parameters

There are many simulation parameter options. We will mainly be concerned with the start and stop times, which tell Simulink over what time period to perform the simulation.

The solver options determine what kind of solver the computer uses as it applies a numerical method to solve the set of ordinary differential equations that represent the model. Simulink provides two main types of solvers —fixed-step and variable-step solvers. Variable-step solvers vary the step size during the simulation. They reduce the step size to increase accuracy when the states of a model change rapidly and during zero-crossing events. They increase the step size to avoid taking unnecessary steps when the states of a model change slowly. Fixed-step solvers, as the name suggests, solve the model at fixed step sizes from the beginning to the end of the simulation. You can specify the step size or let the solver choose it. Generally, decreasing the step size increases the accuracy of the results and increases the time required to simulate the system. In most cases, you will not need to change these settings unless you are told to do so in the manual.

Close the dialog box and run the simulation by clicking on the play icon. By double clicking on the scope block, you can see the system output (the speed) for a step force input. Change the values of b and m for different time constants, see the results and try to interpret them. The larger the time constant, the slower the response.

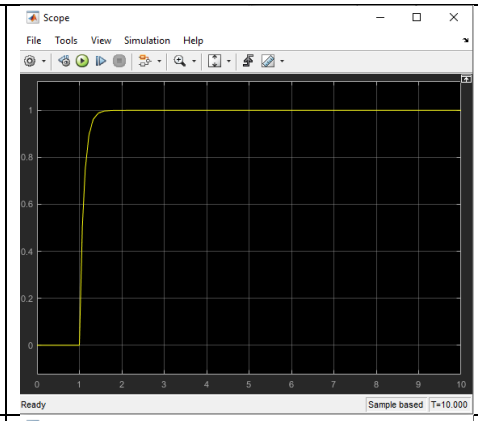
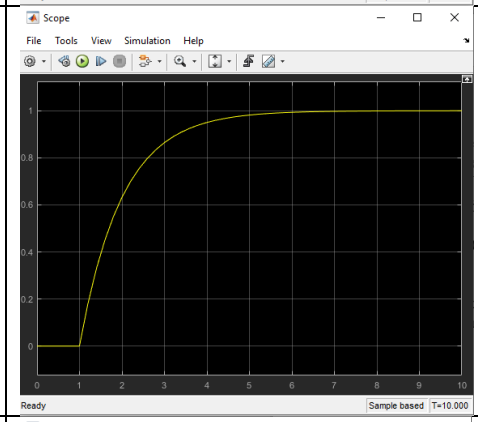
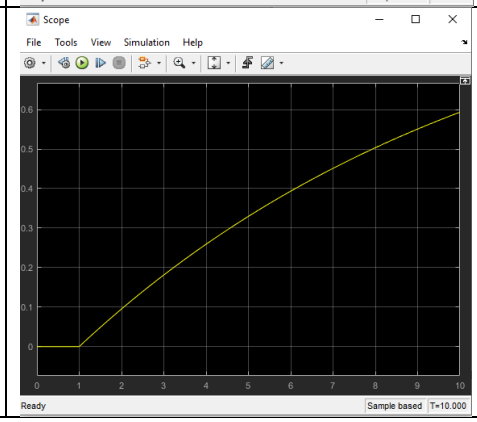
T=0.1	
T=1	
T=10	

Figure 11. First order system response with different time constants

Another example of a first order system analogous to the linear motion above, is the first order rotational motion. For example a circular body of inertia J rotating with damping factor b driven by an outside torque $T(t)$ as shown in the following figure.

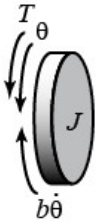


Figure 12. A mass rotating with friction

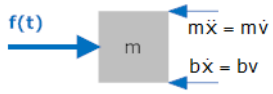
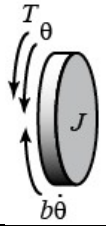
Similar to the linear motion example, the first step is to derive the transfer function between the input and output of the system (torque T and angular speed ω). By writing the differential equation governing the motion and taking the Laplace transform, the transfer function is obtained as follows

$$J\dot{\omega} + b\omega = T(t)$$
$$Js\Omega(s) + b\Omega(s) = T(s)$$
$$\frac{\Omega(s)}{\mathrm{T}(s)} = H(s) = \frac{1}{Js+b} = \frac{\frac{1}{b}}{1+\frac{J}{b}s}$$

In the above equation, $1/b$ is called the static gain and $T = J/b$ is called the time constant.

Example 2: Create a second order system model and simulate it in the time domain.

In the previous example, the relationship between the torque (force) input and the angular (linear) speed as the output translates to a first order transfer function. If we were looking to find the angular (linear) displacement as the output, the system would be a second order one according to the following derivations:
Let us assume that in addition to the friction damping factor b, a spring with the constant k is present.

	
$M\ddot{x} + b\dot{x} + kx = f(t)$ $ms^2X(s) + bsX(s) + kX(s) = F(s)$ $\frac{X(s)}{F(s)} = H(s) = \frac{1}{ms^2+bs+k}$ $= \frac{1}{k}\frac{\frac{k}{m}}{s^2+\frac{b}{m}s+\frac{k}{m}}$	$J\ddot{\theta} + b\dot{\theta} + k\theta = T(t)$ $Js\Theta(s) + b\Theta(s) + k\Theta(s) = T(s)$ $\frac{\Theta(s)}{\mathrm{T}(s)} = H(s) = \frac{1}{Js^2+bs+k}$ $= \frac{1}{k}\frac{\frac{k}{J}}{s^2+\frac{b}{J}s+\frac{k}{J}}$
Figure13.a Linear motion model	Figure13.b Circular motion model

In the above equation, $k_p = 1/k$ is called the static gain, $\omega_n = \sqrt{\frac{k}{J}}\ or\ (\sqrt{\frac{k}{m}})$ is called the natural frequency, and $\zeta = \frac{b}{2\sqrt{kJ}}\ or\ \frac{b}{2\sqrt{km}}$ is called the damping ratio. This means that once a standard unit step input in applied to the system, $1/k$ determines how big the output (angular/linear displacement) is compared to the input (torque/force) and $\frac{m}{b}(\frac{J}{b})$ determines how fast the response is.
Let's simulate the system with $J(or\ m) = b = k = 1$ and obtain the response in Simulink.

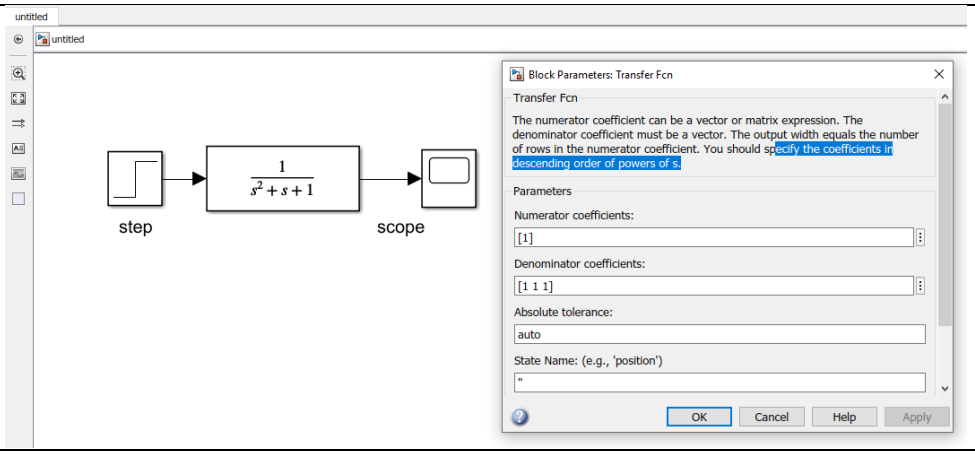


Figure 14. Second order system simulation model

With these parameters, we would have $\omega_n = 1$, $\zeta = \frac{1}{2}$ which according to the following figure, means the response will be an under-damped one.

$\zeta = 0$	$\zeta < 1$	$\zeta = 1$	$\zeta > 1$
undamped	**underdamped**	**critically damped**	**overdamped**

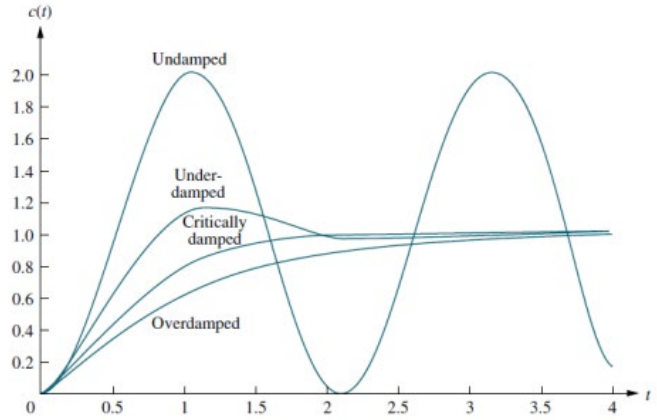
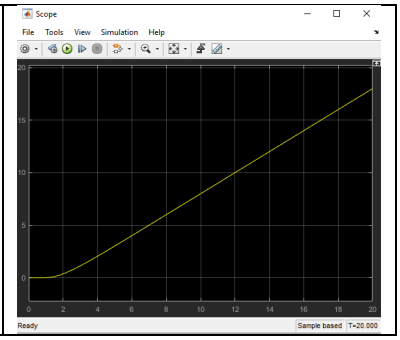


Figure 15. Types of transient response for a second order system (stable)

Change the values of system parameters according to the following table and observe different types of a second order system responses:

$m = 1, k = 0, b = 1$	

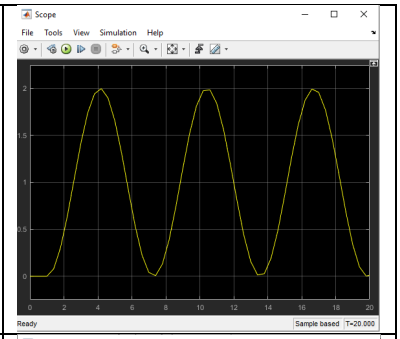
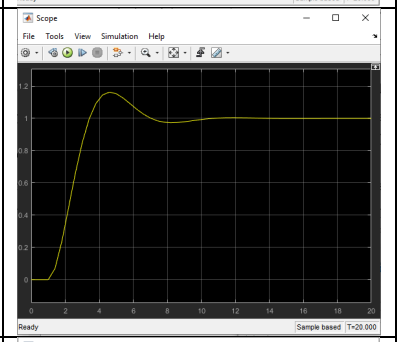
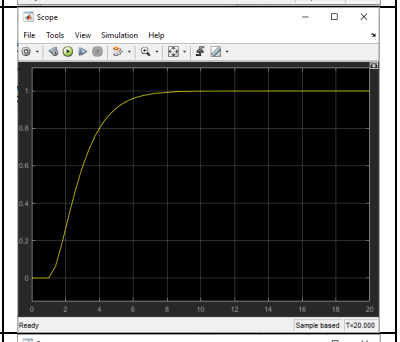
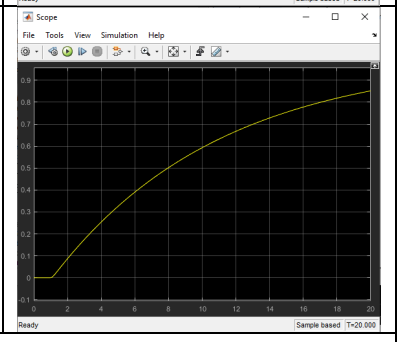
$m = k = 1, b = 0$	
$m = b = k = 1$	
$m = k = 1, b = 2$	
$m = k = 1, b = 10$	

Figure 16. Second order system responses

Thus, the open loop system is either unstable, or stable with response characteristics that might be different from what is needed. This shows the necessity of adding a controller to the system as will be illustrated in the next example. For the system with the underdamped response in the above table, find the transient response characteristics using the following formulas:

t_r rise time: time to rise from 0 to 100% of the steady state response	$t_r = \frac{1}{\omega_n\sqrt{1-\zeta^2}} tg^{-1}\left(\frac{\sqrt{1-\zeta^2}}{\zeta}\right)$

t_p peak time: time required to reach the first peak.	$t_p = \frac{pi}{\omega_n\sqrt{1-\zeta^2}}$
M_p maximum overshoot: the biggest peak	$M_p = \exp(-\frac{\zeta pi}{\sqrt{1-\zeta^2}})$
t_s settling time: time to reach and stay within a 2% (or 5%) tolerance of the final value	$t_s = \frac{4}{\zeta\omega_n}(2\%), \frac{3}{\zeta\omega_n}(5\%)$

Example 3: Design and test a PID controller

In this example a PID controller is used to stabilize an unstable system and force it to show the desired transient response as well.

To create a feedback loop that includes a PID controller, follow these steps:

Disconnect (Delete) the signal connecting the step input to the system transfer function.

Copy and paste or drag and drop the Add block from *Simulink>Math operations*

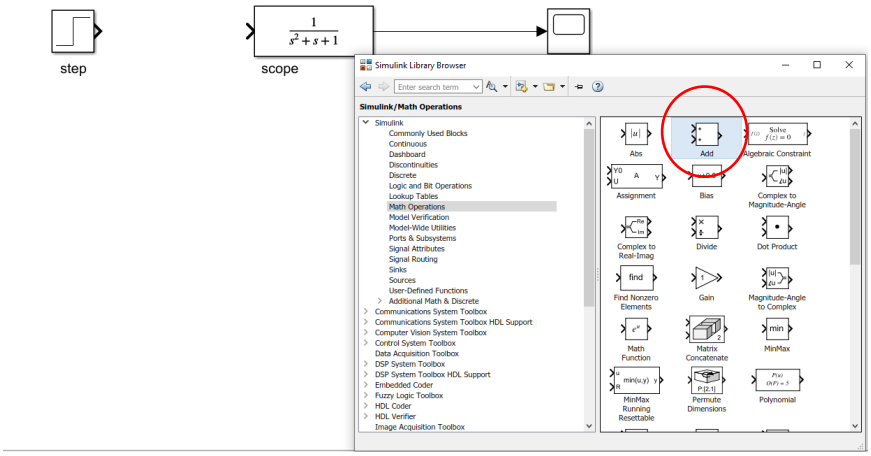


Figure 17. The Add block

Copy and paste or drag and drop the PID Controller block from *Simulink>Continuous*

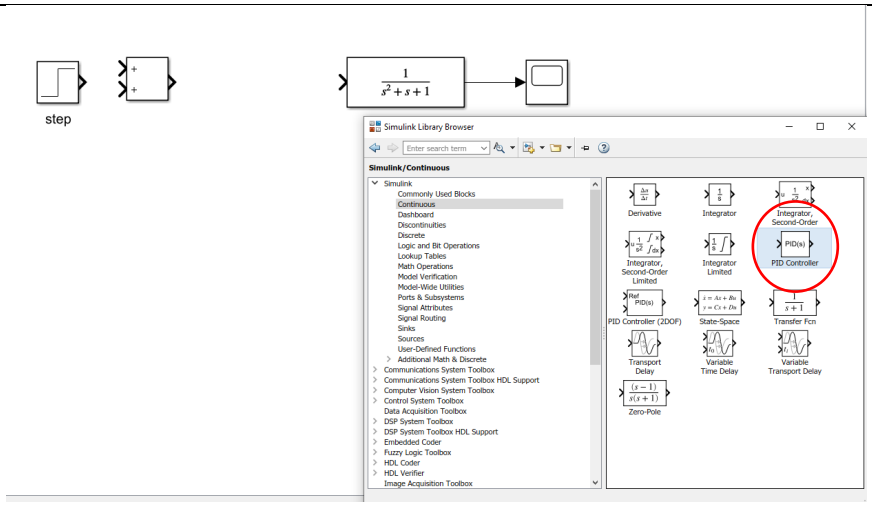


Figure 18. PID Controller Block

Double click on the Add block to see the block parameters and change the sign of the second port of the addition block to – for negative feedback

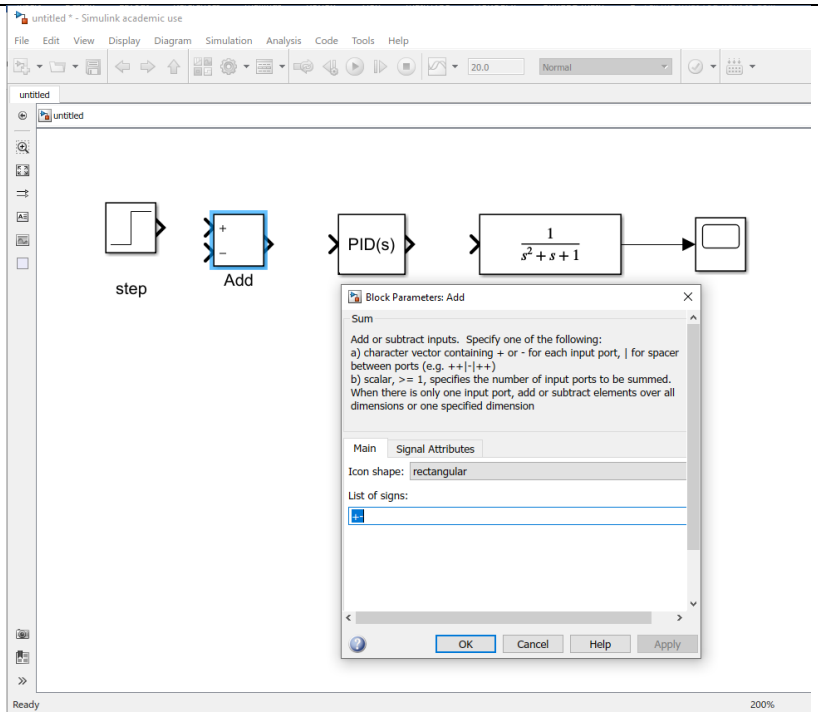


Figure 19. Negative feedback sign insertion

Connect the step input to the addition block, the output of the Add to the input of the PID controller, the output of the PID controller to the input of the transfer function, and the output of the transfer function to the scope for viewing. Then draw a signal line starting from the negative input of the Add block and connect it to the output signal between the transfer function and the scope.

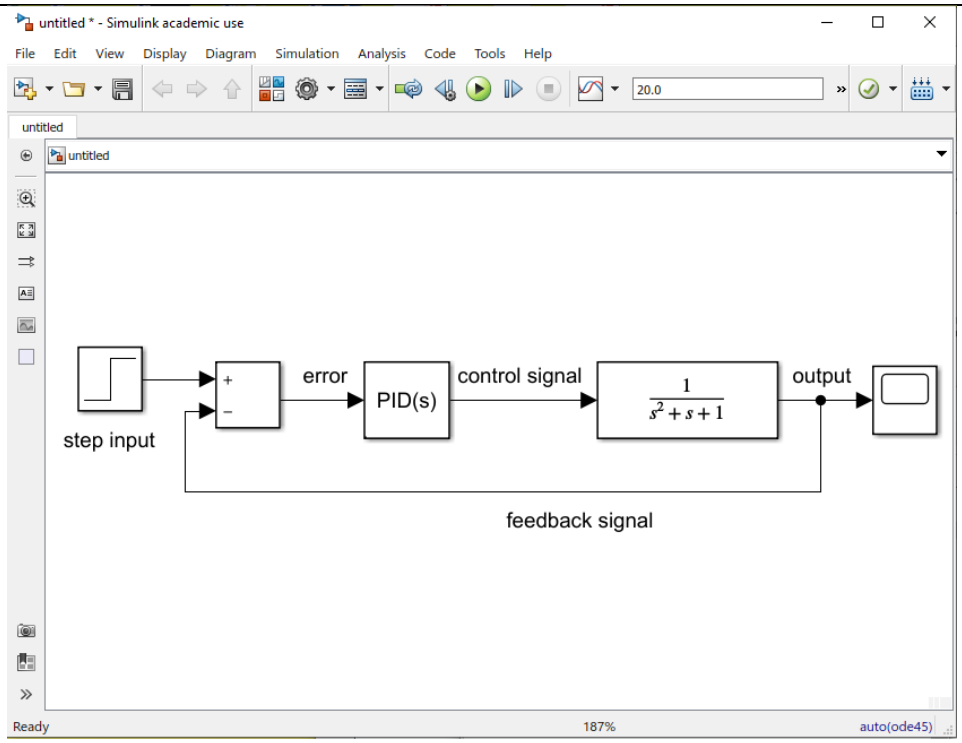


Figure 20. Closed loop system with a PID controller

To set the controller parameters, double click on the PID block to see the block parameters as shown here.

Let us assume that we would like to design a controller for the unstable second order system with the parameters ($m = 1, k = 0, b = 1$). Let us also assume that the design criteria is as follows:

$M_p = 20\%$ maximum overshoot	$\Longrightarrow\Longrightarrow\Longrightarrow \zeta = 0.673$
$t_s = 5s$ settling time (5%)	$\Longrightarrow\Longrightarrow\Longrightarrow \omega_n = 0.89$

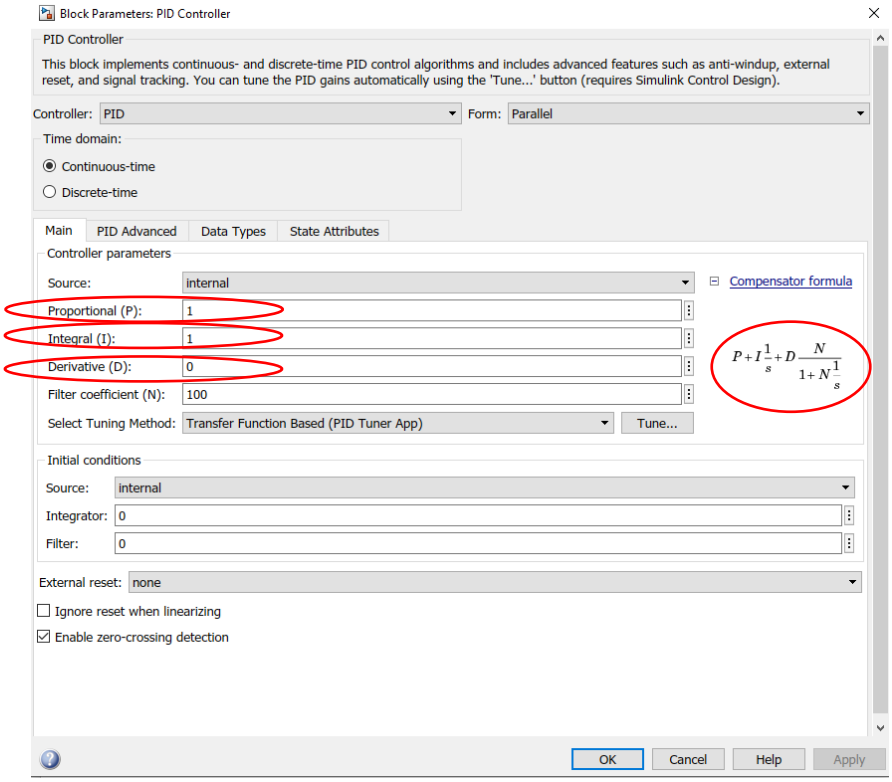


Figure21. PID controller tuning parameters

As you can see in the block parameters window, the (ideal) transfer function for the controller is $P + I\frac{1}{s} + Ds = \frac{Ps^2+Is+D}{s}$. Since the ideal derivative term Ds is non-causal (derivative means prediction of future values which impossible in practice), it is replaced by a filter that simulates the derivational behavior with a causal transfer function $s \cong \lim_{N\to\infty} D\frac{N}{1+N\frac{1}{s}}$.

You can insert the P, I, D coefficients for the controller in this block after you have designed them to obtain your desired response. The system transfer function is:

$$G(s) = \frac{1}{s^2 + 1}$$

The controller Transfer function:

$$C(s) = \frac{Ps^2 + Is + D}{s}$$

The closed loop transfer function:

$$H(s) = \frac{Ps^2 + Is + D}{S^3 + (P+1)s^2 + Is + D}$$

The denominator of the closed loop transfer function is the characteristic equation of the system. With three controller parameters, and a 3rd order closed-loop system, the poles can be assigned. To do this, use the (ζ, ω_n) from your design criteria for a second order system, along with a 3rd

pole of your own choosing at an arbitrarily far left position $-\alpha\omega_n$, we set the characteristic equation to be

$$s^3 + ((2\zeta + \alpha)\omega_n)s^2 + \left((2\zeta\alpha + 1)\omega_n^2\right)s + \alpha = 0$$

By comparing this with the system characteristic equation and choosing $\alpha = 10$, the following PID parameters are obtained:

$$M_p = 20\% \Longrightarrow \zeta = 0.673$$

$$t_s = 5s \Longrightarrow \omega_n = 0.89$$

$$choosing\ \alpha = 10$$

$$P = -1 + 2\zeta + \alpha = 10.346$$

$$I = 2\zeta\alpha + 1 = 14.46$$

$$D = \alpha\omega_n^3 = 7.05$$

The resulting PID controller should result in a response similar to the following:

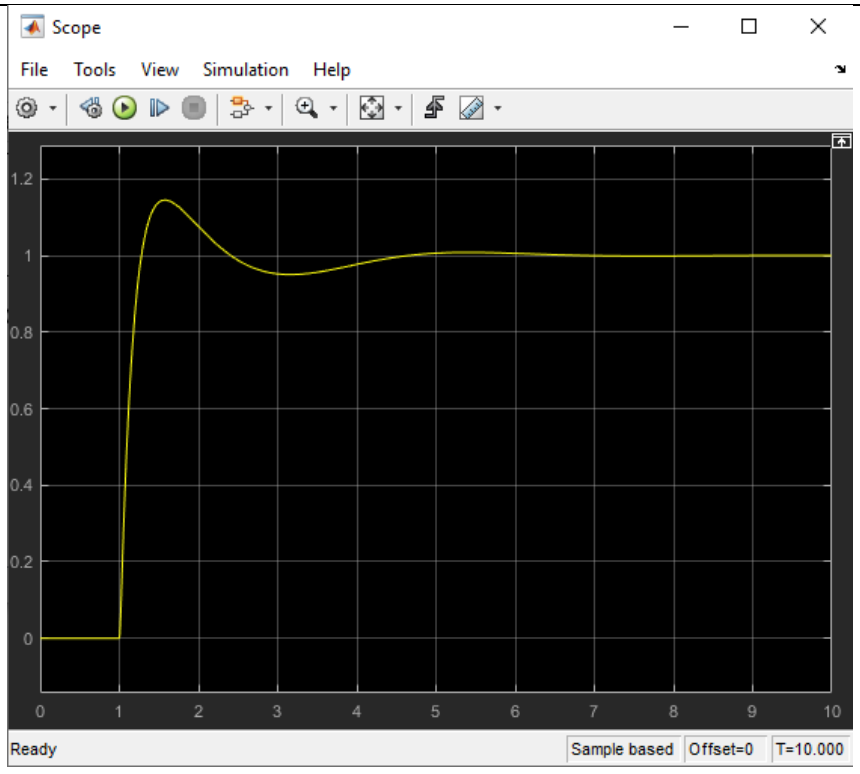


Figure 22. Closed loop response of the controlled system

Note that there are multiple ways to tune a PID controller and you might be asked to use a different one depending on the system you will be working with.